4107 Assignment 1

This assignment implements an Information Retrieval (IR) system based on the vector space model for a collection of scientific documents. The system processes a corpus and a set of test queries, builds an inverted index, and performs retrieval and ranking using a TF-IDF weighted vector space model with cosine similarity.

Two different runs were executed:

1. Title Only: Documents are represented using only the title field.
2. Title + Full Text: Documents are represented using the concatenation of the title and the full text.

The results of both runs are evaluated with *trec_eval*.

Identification

Group 33: Yucheng Chen (300194614), Junyang Wang (300241369), Danning Chen (300234800)

Task distribution:

Name	Responsibility
Danning Chen	File parsing, Text preprocessing (Part I)
Junyang Wang	Indexing (Part II), Document writing
Yucheng Chen	Retrieval and Ranking (Part III), Testing

How to run

1. Install python 3.12 or above
2. Install NLTK:

   For Linux/MacOS (might need to install pip first if not already do so):

   ```
   pip install --user -U nltk
   ```

   For Windows: Check out here
3. Install Scikit-learn:

   ```
   pip install -U scikit-learn
   ```
4. Install NLTK 'punkt_tab' and 'stopwords' data

   Open python terminal and run following commands:

```
>>> import nltk
>>> nltk.download('punkt_tab')
>>> nltk.download('stopwords', quiet=True)
```

5. Compile and run src/main.py

Note

The result of computing MAP with trec_eval are stored in *MAP_score.txt*. To compute MAP score yourself, check out this guide for installation and usage: https://aldolipani.com/trec_eval-installation-usage-and-behaviour/

Algorithms, Data Structures, and Optimizations Used

1. **Tokenization and Text Cleaning:** We implemented a preprocessing function (preprocess_text) that converts the text to lowercase and removes punctuation using regular expressions. We also use a helper function (safe_join) to handle both list and string types safely.
2. **Inverted Index**: Maps each token to a list of document IDs where the token appears, along with its term frequency (d_f and tf_i).
3. **Dictionary for Documents and Queries:** Each document is stored in a dictionary, mapping IDs to their processed text representation.
4. **Uniform Preprocessing:** Used hash tables for fast lookups and insertions. Stored TF directly in the index to avoid recalculating it during retrieval. Indexed documents incrementally, processing one document at a time to reduce memory consumption.
5. **Stop word:** Implemented stop word removal to eliminate frequent but non-informative words.
6. **TF-IDF weighting + Cosine Similarity ranking:** We constructed our weighting based on following formula while using *TfidfVectorizer* from *Scikit-learn* for efficient TF-IDF computation.

   $$w_{ij} = tf_{ij} * idf_i$$

   The weighting was used to rank documents based on their similarity to the query. Constructed a global TF-IDF matrix to avoid redundant computations and utilize *NumPy* operations for efficient similarity calculations and sorting.

First 10 Answers for the First 2 Queries

Query 0: "0-dimension biomateri lack induct properti."

0 Q0 42421723 1 0.1037 run_time

0 Q0 35008773 2 0.0922 run_time

0 Q0 994800 3 0.0855 run_time

0 Q0 26731863 4 0.0850 run_time

0 Q0 12156187 5 0.0800 run_time

0 Q0 14827874 6 0.0745 run_time

0 Q0 13231899 7 0.0732 run_time

0 Q0 18953920 8 0.0717 run_time

0 Q0 7581911 9 0.0693 run_time

0 Q0 21257564 10 0.0669 run_time

Query 2: "1 in 5 million in uk have abnorm prp posit."

2 Q0 13734012 1 0.3156 run_time

2 Q0 17333231 2 0.2836 run_time

2 Q0 42240424 3 0.2822 run_time

2 Q0 13770184 4 0.2405 run_time

2 Q0 18617259 5 0.1510 run_time

2 Q0 695938 6 0.1423 run_time

2 Q0 1292369 7 0.1265 run_time

2 Q0 32481310 8 0.1243 run_time

2 Q0 17415081 9 0.1194 run_time

2 Q0 3716075 10 0.1190 run_time

MAP-Score

We evaluated our system using trec_eval for two configurations:

1. Using only titles (Results_title_only.txt)
2. Using both titles and full text (Results_title_full.txt)

Configuration	Score
Title	0.3867
Title + Text	0.5430

The increase in recall suggests that full-text queries provided more context, enabling better ranking of relevant documents. Precision at top 5 and top 10 documents also increased, meaning the top-ranked documents were more likely to be relevant.

```
(base) yuchengchen@ChendeMacBook-Air src % trec_eval qrels.txt
Results_title_only.txt
```

```
runid                   all run_time
num_q                   all 300
num_ret                 all 29929
num_rel                 all 339
num_rel_ret             all 244
map                     all 0.3867
gm_map                  all 0.0174
Rprec                   all 0.2947
bpref                   all 0.7058
recip_rank              all 0.4016
iprec_at_recall_0.00    all 0.4019
iprec_at_recall_0.10    all 0.4019
iprec_at_recall_0.20    all 0.4019
iprec_at_recall_0.30    all 0.3966
iprec_at_recall_0.40    all 0.3900
iprec_at_recall_0.50    all 0.3893
iprec_at_recall_0.60    all 0.3777
iprec_at_recall_0.70    all 0.3769
iprec_at_recall_0.80    all 0.3764
iprec_at_recall_0.90    all 0.3759
iprec_at_recall_1.00    all 0.3759
P_5                     all 0.1073
P_10                    all 0.0623
P_15                    all 0.0447
P_20                    all 0.0347
P_30                    all 0.0242
P_100                   all 0.0081
P_200                   all 0.0041
P_500                   all 0.0016
P_1000                  all 0.0008
(base) yuchengchen@ChendeMacBook-Air src % trec_eval qrels.txt
Results_title_full.txt
runid                   all run_time
num_q                   all 300
num_ret                 all 29929
num_rel                 all 339
num_rel_ret             all 303
map                     all 0.5430
gm_map                  all 0.1339
Rprec                   all 0.4344
bpref                   all 0.8931
recip_rank              all 0.5567
iprec_at_recall_0.00    all 0.5568
iprec_at_recall_0.10    all 0.5568
iprec_at_recall_0.20    all 0.5568
iprec_at_recall_0.30    all 0.5559
iprec_at_recall_0.40    all 0.5475
iprec_at_recall_0.50    all 0.5475
iprec_at_recall_0.60    all 0.5355
iprec_at_recall_0.70    all 0.5328
iprec_at_recall_0.80    all 0.5318
iprec_at_recall_0.90    all 0.5312
iprec_at_recall_1.00    all 0.5312
P_5                     all 0.1473
P_10                    all 0.0823
P_15                    all 0.0576
```

```
P_20                    all 0.0443
P_30                    all 0.0313
P_100                   all 0.0101
P_200                   all 0.0050
P_500                   all 0.0020
P_1000                  all 0.0010
```